This is where the architecture differs significantly from charts. While charts are often data structures rendered by frontend libraries, **images are almost always generated by distinct AI models** (like Stable Diffusion or DALL-E) that must be orchestrated by the primary chat model.

Here are the answers to your two specific questions regarding image integration and display.

1. Image Model Integration: Tool vs. Toggle

The answer is: **Both are used depending on the platform's complexity.**

However, the industry standard for robust, multi-modal chat assistants (like ChatGPT or Claude) is to treat the image generator as a **tool call** initiated by the LLM.

Approach A: The LLM as an Orchestrator (Tool Call)
In this modern architecture, the LLM acts like a project manager. It does not draw the image itself; instead, it decides *if* an image is needed and *how* it should look.

1. **The User Prompt:** You ask: *"Create a picture of a futuristic cyberpunk cat."*
2.
3. **The LLM Thought Process:** The LLM analyzes the intent. It recognizes it lacks the capability to generate pixels.
4.
5. **The Tool Call:** Instead of outputting text, the LLM outputs a structured JSON tool call internally: `{"tool": "image_generator", "arguments": {"prompt": "A highly detailed, neon-lit cyberpunk cat sitting on a rainy Tokyo rooftop."}}`.
6.
7. **Execution:** The application backend intercepts this JSON, sends the prompt to an image model API (like DALL-E 3 or Stability AI), and receives back a URL pointing to the generated PNG or JPEG.
8.
9. **Display:** The chat UI renders this URL as an image tag (`<img>`).
10.

Approach B: The User Toggle (Explicit Separation)
In simpler or specialized applications, the intelligence may be separated entirely from the UI level.

- **Example:** You are on a platform that has a "Chat Tab" and an "Image Gen Tab." If you are on the Image Gen tab, your prompt goes directly to a specialized diffusion model frontend. The main conversational LLM is bypassed entirely. The platform might show the output image in the chat history later, but the conversational model was not involved in requesting it.
-

2. Rendering the Image (From API to UI)

If an API call (tool execution) returns an image URL (e.g., `[https://oaidalleapiprodscus.blob.core.windows.net/.../img-xyz.png](https://oaidalleapiprodscus.blob.core.windows.net/.../img-xyz.png)`), how does it appear in the chat?

It is **not a simple stream**. It requires an extra step by the frontend to ensure a smooth user experience.

Here is the lifecycle of an image rendering in a modern chat UI:

Step 1: The "Thinking" Phase
While the image model is generating the image (which can take 5 to 30 seconds), the LLM usually streams back a placeholder text response, like *"Sure, give me a moment while I generate that image..."*. The UI shows a loading spinner or a shimmering skeleton loader where the image will eventually appear.

Step 2: The Atomic URL Injection
Once the image generation API completes and returns the final URL to your backend, your chat application's backend updates that specific message bubble in its database, replacing the "loading state" with the actual final URL.

Step 3: The Frontend Fetch and Display
The frontend application receives the updated message payload containing the image URL. It then immediately triggers a standard browser fetch operation for that URL.

This is handled by the standard HTML/JavaScript `<img>` tag lifecycle:

1. The browser sees the `<img src="[THE_IMAGE_URL.png]">` tag added to the DOM.
2.
3. The browser initiates an asynchronous HTTP `GET` request to the image hosting location (e.g., the OpenAI or AWS blob storage server).
4.
5. While the image bytes are downloading over the network, the browser displays the default broken image icon or the loader you styled.
6.
7. Once the complete PNG file finishes arriving, the browser performs a "layout shift" and renders the finalized pixel data inside the chat bubble.
8.

Open-Source Projects and Ecosystems

To see how developers implement image generation tooling and rendering in local or self-hosted environments, explore these projects:

- `open-webui / open-webui` **(GitHub):** This is the leading open-source interface for Ollama. It has deep, built-in support for image generation tools. You can enable image generation (using Automatic1111 or ComfyUI backends). When you ask for an image, the LLM identifies the tool call, the interface displays a spinner, and the resulting URL is rendered inline.
-
- `AUTOMATIC1111 / stable-diffusion-webui` and `comfyanonymous / ComfyUI` **(GitHub):** These are the actual backend engines used to *create* the images. While they have their own UIs, they expose robust APIs that chat interfaces (like Open WebUI) plug into to function as the "tool" the LLM calls.

-
- danny-avila / LibreChat **(GitHub):** A massive multi-provider chat UI. It allows configuration of various image generation endpoints (DALL-E, Google Imagen, and local SD backends), unifying their display within the chat history.
-
- lobehub / lobe-chat **(GitHub):** A sleek chat UI framework that features "Plugins." Image generation is treated as a plugin tool call, orchestrated by the primary AI agent.